

NATIVE + CLOUDFLARE WORKERS

Akamata

Portable Zig

Backend Framework

普通のZig関数から、型付きWeb APIへ。

Zigで書くアプリケーション

Zigでbackendを作る。

NativeかCloudflare Workersを選ぶ。

Application

Native

VPS / local / Container

Cloudflare Workers

WASM / Edge

普通の関数がWeb APIになる

```
fn hello() []const u8 {  
    return "Hello, Akamata!";  
}
```

```
pub const Application = ak.App(.{  
    .routes = .{  
        ak.get("/", hello),  
    },  
});
```

tests/docs/minimal.zigのApplication宣言。allocatorとentry pointは省略。

Zigの型をHTTP contractへ

Zig function + struct + error set

明示的な入力

Path / Query / Header / Cookie
JSON DTO / Validation / Principal

型付きの結果

Response / HTTP error mapping
Security / Capability metadata

Router / OpenAPI / TypeScript client / Tests

handlerと明示endpoint metadataを、既存HTTP coreで共有。

小さなAPIから実アプリケーションへ

01 Function

02 Typed JSON API

03 Database

04 Authentication

05 Storage / Queue

06 Realtime

最初はContextなしで開始。必要になればContext、middleware、provider ownerへ降りられます。

共通applicationと、明示的なprovider

Application / Portable Contract

Native

SQLite / Turso

Filesystem

jobs.Queue

Native realtime Service

Workers

D1 / Turso

R2

Queues

Durable Objects gateway

ContainerはNative providerを利用。entry point・owner・platform制限は明示します。

Native Threadedの測定結果

165k

/hello

HTTP 200 req/s

164k

/echo

HTTP 200 req/s

87k

/db/1

HTTP 200 req/s

ローカルruntimeの測定値。他frameworkとの順位・比較を示すものではありません。

macOS 27 / Apple M2 / Zig 0.17.0 / ReleaseFast / Threaded

32 keep-alive / loopback / oha 1.15.0 / 3 × 3s per workload / 全roundの平均

[raw data](#) · [commit/binary SHA](#) · [再現手順](#)

domain codeをEdgeへ

Workers WASM + managed JavaScript glue

D1 R2 Queues Durable Objects

DTO・errorの意味・event protocol・portable serviceのeffectsを共有。
targetに合わせてbindingとownerを設定します。

adapter/host contractはoffline検証。live Cloudflare認証済みとは主張しません。

socketなしのapplication test

```
var client = app.client(allocator);  
var response = try client.post("/users")  
    .json(.{ .name = "Alice" }).send();  
defer response.deinit();  
try response.expectStatus(.created);
```

MemoryStore / QueueRecorder / Principal injection

共通DX fixtureをNative / Workers WASMで実行。fake providerはlive deliveryの証明ではありません。

開発の流れ

init

最小app

dev

build / run / watch

test

appをテスト

check

projectを検証

deploy

targetを明示

akamata init myapp

appの成長に合わせてroutes・OpenAPI・capabilitiesを確認。

commandはproject設定を利用。deployはresourceを暗黙作成しません。

deploymentまで追跡するPortable Contract

Application Capability Provider Binding Owner Deployment

必要な機能を宣言。

targetがどう提供するかを検証。

local validation · remote reachability · application readinessは異なる証拠レベルです。

Native / Workers / Containerの対応

	Native	Workers	Container
HTTP	Threaded	WASM / fetch	Threaded
DB	SQLite / Turso	D1 / Turso	SQLite / Turso
Storage	Filesystem	R2	Filesystem *
Queue	jobs.Queue	Queues	jobs.Queue
Realtime	Native Service	DO gateway	Native Service

* DB/filesystemの永続化にはvolumeが必要。streaming・retry・gatewayの詳細はplatformで異なります。

最初のアプリケーション

```
akamata init myapp
```

```
cd myapp
```

```
akamata dev
```

[Getting Started](#) [Guides / Examples](#) [GitHub](#)

先にCLIをinstall。mainのAPIはGetting Startedのlocal dependency overrideで試せます。